

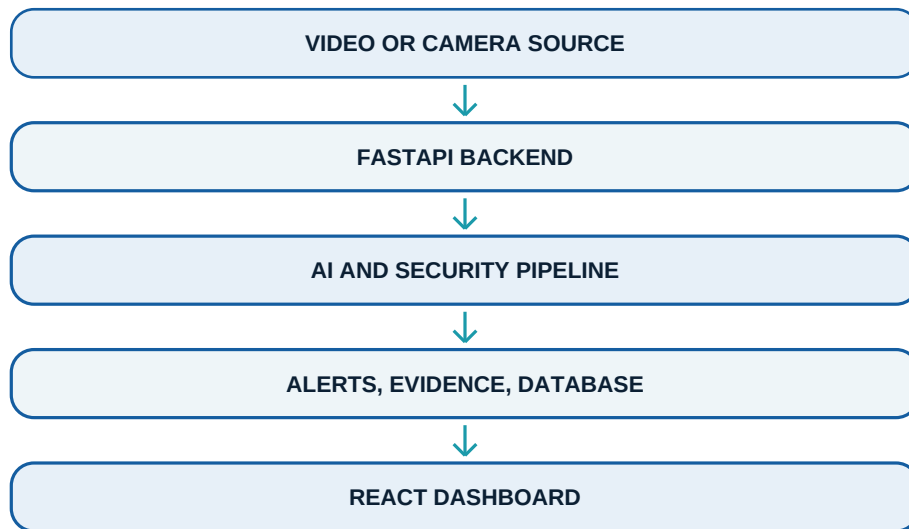
SENTINEL AI

BORDER CONTROL UNIT

Complete Beginner-Friendly Technical Guide

SIH 2026 | Problem Statement: SIH26187

A code-accurate tour of the current Sentinel AI MVP: from a video frame, through OpenCV, YOLO, IoU tracking, zones, threat scoring, alerts, evidence, SQLite, WebSockets, and the React dashboard.



Reading rule

This guide documents the current repository as inspected. Items that are inactive, UI-only, or future work are labelled instead of being presented as working features.

Contents. Contents

The page numbers are generated from the headings in this guide.

Contents. Contents	2
1. Cover and scope	3
2. What are we actually building?	4
3. Absolute basics	6
4. Actual technology stack	7
5. Complete system architecture	8
6. Project folder structure	9
7. Frontend explained	10
8. Backend explained	11
9. The AI and computer-vision pipeline	13
10. YOLO explained from zero	14
11. Object tracking and IoU	15
12. Zone detection	16
13. Threat engine	17
14. Alert system	18
15. WebSockets	19
16. Database	20
17. Video processing	21
18. Evidence and event storage	22
19. Complete end-to-end example	23
20. Code walkthrough	24
21. How everything connects	26
22. What happens when I start the project?	28
23. Common terms glossary	29
24. Why this architecture?	30
25. Limitations of the current MVP	31
26. Future improvements	32
27. SIH demo explanation	33
28. Explain it in 60 seconds	34
29. If a judge asks how it works	35

1. Cover and scope

This document is the technical textbook for the Sentinel AI Border Control Unit MVP. It explains the implementation that exists in the repository at generation time, including the active runtime path and the important limitations discovered during inspection.

Implemented versus planned

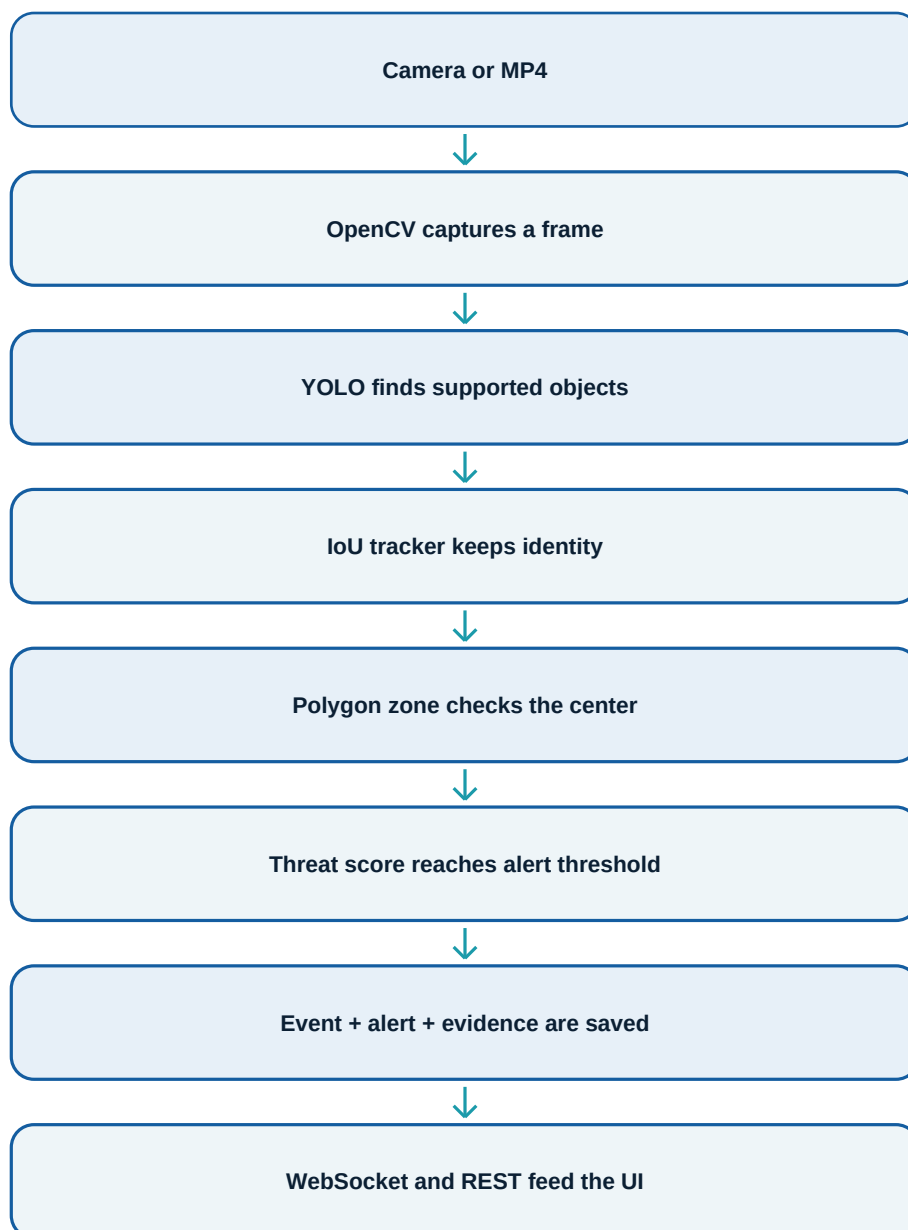
The backend, React interface, local operator authentication, real YOLO model integration, video worker, zone APIs, alert persistence, evidence serving, and WebSocket broadcast are implemented. Runtime detection settings are available through the authenticated settings API. Several older page and service files exist but are not the active routed implementation.

2. What are we actually building?

Sentinel AI is a local border-surveillance system. It watches a configured video source, looks for supported objects, gives a continuing track number to objects that overlap across frames, checks whether their center point lies inside a camera-specific polygon, and turns a persistent intrusion into a stored alert.

The real-world story

Imagine a camera watching a restricted section of a border facility. A person appears. One frame is not enough to decide that the person is a threat, so the system keeps processing frames. YOLO proposes a person box, the tracker keeps the same track ID when the next box overlaps, the zone detector checks the box center, and the threat engine waits for five matching hits. For a person, the persistence rule raises the score from 45 to 65, which crosses the active-stream alert threshold of 61. The backend then records an event and alert, saves an annotated JPEG, broadcasts a JSON alert, and the dashboard can show a notification.



Who uses it?

The current UI is designed for an operator who needs a quick view of camera sources, a live processed preview, active alerts, evidence, historical events, analytics totals, and zone configuration. The system is a local MVP, not a production command-and-control system.

3. Absolute basics

Software is a set of instructions that a computer executes. A program is a packaged set of those instructions. In Sentinel AI, different programs cooperate: a browser runs the frontend, Python runs the backend, and OpenCV plus YOLO perform visual work.

Term	Plain-language meaning	Sentinel AI analogy
Frontend	The part the operator sees and clicks.	Control-room screen in React.
Backend	The program that receives requests and performs work.	The local supervisor and processing brain.
Server	A program waiting for requests on an address and port.	Uvicorn serves FastAPI on port 8000.
API	A defined way for two programs to communicate.	React calls paths such as <code>/api/cameras</code> .
Database	An organized, searchable collection of stored records.	SQLite stores cameras, detections, events, zones, and alerts.
AI / machine learning	A model uses learned patterns rather than hand-written rules alone.	YOLO recognizes visual object classes.
Computer vision	Software that extracts meaning from images or video.	OpenCV reads frames and draws overlays.
Video frame	One still image from a video stream.	Each captured frame enters the pipeline.
HTTP request	One message asking a server to do or return something.	<code>GET /api/alerts</code> asks for stored alerts.
JSON	A text format for structured data.	Camera and alert API responses use JSON.
WebSocket	A connection kept open for two-way messages.	The backend pushes new alerts to the browser.

A small analogy

The frontend is a control-room monitor. The backend is the operator who handles requests. The API is the radio protocol. The database is the incident notebook. A WebSocket is an open radio channel. YOLO is a visual assistant that points out likely objects, while the threat engine applies the site rules.

4. Actual technology stack

Technology	Where used	Why it is here	Simple explanation
Python	backend/	Runs the API, workers, AI pipeline, and database access.	A general-purpose programming language.
FastAPI	backend/app/main.py and app/api/	Defines HTTP and WebSocket endpoints.	A Python web framework.
Uvicorn	Startup command	Runs the FastAPI application as a local server.	The server process that hosts FastAPI.
SQLAlchemy	backend/app/database/	Maps Python classes to SQLite tables.	A database access layer.
SQLite	sentinel.db	Stores local MVP records without a separate database server.	A file-based relational database.
OpenCV	app/video/ and app/ai/	Reads frames, encodes JPEGs, draws boxes, and tests polygons.	A computer-vision toolkit.
Ultralytics YOLO	app/ai/detector.py	Loads model.pt and performs object inference.	A real-time object detector.
WebSockets	app/api/websocket.py and frontend hook	Pushes an alert when it is created.	A persistent live connection.
React 19	frontend/src/App.jsx	Renders the operator interface from components and state.	A library for interactive browser interfaces.
Vite	frontend/vite.config.js and package scripts	Runs the frontend development server and build.	A fast frontend toolchain.
React Router	frontend/src/App.jsx	Maps browser paths to local components.	The frontend page navigator.
lucide-react	frontend/src/App.jsx	Supplies interface icons.	A React icon library.
python-dotenv	backend/app/main.py	Loads optional environment values.	Reads .env-style configuration.
fetch	frontend/src/services/api.js	Sends REST requests from the browser.	The browser's HTTP client.

Dependency note

axios appears in frontend/package.json, but the active API helper uses the browser fetch function. Tailwind is not in the active stack; the interface uses App.css and index.css.

5. Complete system architecture

The runtime is not a single linear request. The browser uses REST to configure and read data, while a background stream worker continuously captures and processes frames after a stream is started. A separate WebSocket connection lets the backend push newly created alerts.



How the arrows should be read

- The frontend does not run YOLO. It requests a stream start and displays the MJPEG response.
- The capture reader and inference worker are separate threads. A one-item queue favors the newest frame so processing does not accumulate an unbounded backlog.
- The backend writes the processed JPEG to the current in-memory stream state. The MJPEG route yields that latest image repeatedly.
- The alert path is synchronous inside the worker: database rows and evidence are created before the WebSocket broadcast is scheduled.

6. Project folder structure

The following is the useful runtime tree. Generated bytecode, database files, videos, model weights, and the Git directory are omitted from the teaching tree.

```
Sentinel_Ai/
|-- backend/
|   |-- main.py           Uvicorn entry point from backend/
|   |-- app/main.py       FastAPI app and router registration
|   |-- app/api/          REST, stream, zone, and WebSocket routes
|   |-- app/ai/           detector, tracker, zones, threat, pipeline
|   |-- app/video/        stream worker, MJPEG, finite video helper
|   |-- app/services/     camera, alert, event, analytics services
|   |-- app/database/     SQLAlchemy models, schemas, database setup
|   |-- app/core/         config, logging, security helpers
|   |-- tests/            backend automated tests
|-- frontend/
|   |-- src/App.jsx       active routed UI and runtime orchestration
|   |-- src/services/api.js active REST client
|   |-- src/hooks/useWebSocket.js active alert WebSocket hook
|   |-- src/components/zones/ active ZoneEditor
|   |-- src/pages/       older/inactive page modules in this revision
|   |-- package.json     npm scripts and frontend dependencies
|-- ai_models/yolo/model.pt active stream-worker model path
|-- data/videos/         video sources
|-- data/evidence/       generated evidence JPEGs
|-- sentinel.db          project-root SQLite database
|-- docs/               existing documentation and this generator
|-- README.md           project setup and API overview
```

Important startup files

File	What it does	When it runs
backend/main.py	Imports app from app.main.	When Uvicorn is run from backend/.
backend/app/main.py	Creates FastAPI, CORS, database tables, routers, and /evidence.	At backend import/startup.
frontend/src/main.jsx	Bootstraps React and imports App.css/index.css.	In the browser entry bundle.
frontend/src/App.jsx	Defines the current Shell, pages, stream supervisor, and routes.	For every browser session.
backend/app/video/stream_manager.py	Owns active camera worker state and latest frames.	After POST /api/streams/start.

7. Frontend explained

React lets the frontend describe what the screen should look like for the current state. A component is a reusable function that returns interface markup. State is a value that can change, such as whether data is loading, which camera is selected, or whether an alert has been acknowledged.

The active frontend pattern

```
function getManualStops() {
  try { return new Set(JSON.parse(window.sessionStorage.getItem(MANUAL_STOPS_KEY) || "[]").map(Number)); } catch { return new Set() }
}

function setManualStop(cameraId, stopped) {
  const stops = getManualStops();
  if (stopped) stops.add(Number(cameraId)); else stops.delete(Number(cameraId));
  window.sessionStorage.setItem(MANUAL_STOPS_KEY, JSON.stringify([...stops]));
}

function cameraStatus(camera) {
```

The active App.jsx has a small useRemote helper. It calls a REST loader, stores loading/error/data state, and exposes reload. The Dashboard, Cameras, Monitoring, Alerts, Events, and Analytics components use this pattern. The `asArray` normalizer makes an array response safe for the table views.

Active routes

Path	Active component	What the operator can see or do
/login	Login	Authenticates an operator through /api/auth/login before entering the protected UI.
/ and /dashboard	Dashboard	Camera tiles, metrics, active alerts, live alert notification.
/monitoring	Monitoring	Select camera, start/stop stream, live processed preview, zone editor.
/alerts	Alerts	REST list of stored alert rows and evidence links.
/alerts/:alertId	AlertDetailPage	Review details; confirm plays a laptop Web Audio alarm; decline does not.
/events and /archive	Events	Stored Event rows.
/cameras	Cameras	List, add, and delete configured cameras.
/border-map	BorderMap	Markers for cameras with latitude and longitude.
/analytics	Analytics	Counts from /api/analytics and /api/detections.
/settings	SettingsPage	Authenticated runtime thresholds and filter switches; changes apply to new workers.

Inactive or legacy frontend files

`frontend/src/pages/Dashboard.jsx` currently returns null. `frontend/src/hooks/useCameras.js` is a separate hook that calls the same active API helper, but App.jsx does not import it. `frontend/src/services/cameraService.js`, alertService.js, and eventService.js are empty in the inspected revision. The repository contains page/component files that are not selected by the current App.jsx route table.

8. Backend explained

Python is the language used for the server and processing pipeline. FastAPI turns Python functions into network endpoints. A route is a method-plus-path such as GET `/api/cameras`. A request carries input from the browser; a response carries JSON or a stream back.

Router registration

```
from .core.security import get_current_operator, get_stream_operator

load_dotenv()

app = FastAPI(
    title="Sentinel AI Border Control Unit",
    version="0.1.0",
    description=(
        "AI-powered intelligent border surveillance "
        "and video analytics platform."
    ),
)

# -----
# CORS
# -----

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "http://127.0.0.1:5173",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# -----
# Database
# -----

Base.metadata.create_all(bind=engine)
ensure_compatibility_columns()
reset_runtime_statuses()
EVIDENCE_DIR = PROJECT_ROOT / "data" / "evidence"
EVIDENCE_DIR.mkdir(parents=True, exist_ok=True)

# -----
# API Routers
# -----

app.include_router(cameras_router, dependencies=[Depends(get_current_operator)])
app.include_router(auth_router, prefix="/api")
app.include_router(cameras_router, prefix="/api", dependencies=[Depends(get_current_operator)])
app.include_router(alerts_router, prefix="/api", dependencies=[Depends(get_current_operator)])
app.include_router(events_router, prefix="/api", dependencies=[Depends(get_current_operator)])
app.include_router(detections_router, prefix="/api", dependencies=[Depends(get_current_operator)])
app.include_router(analytics_router, prefix="/api", dependencies=[Depends(get_current_operator)])
```

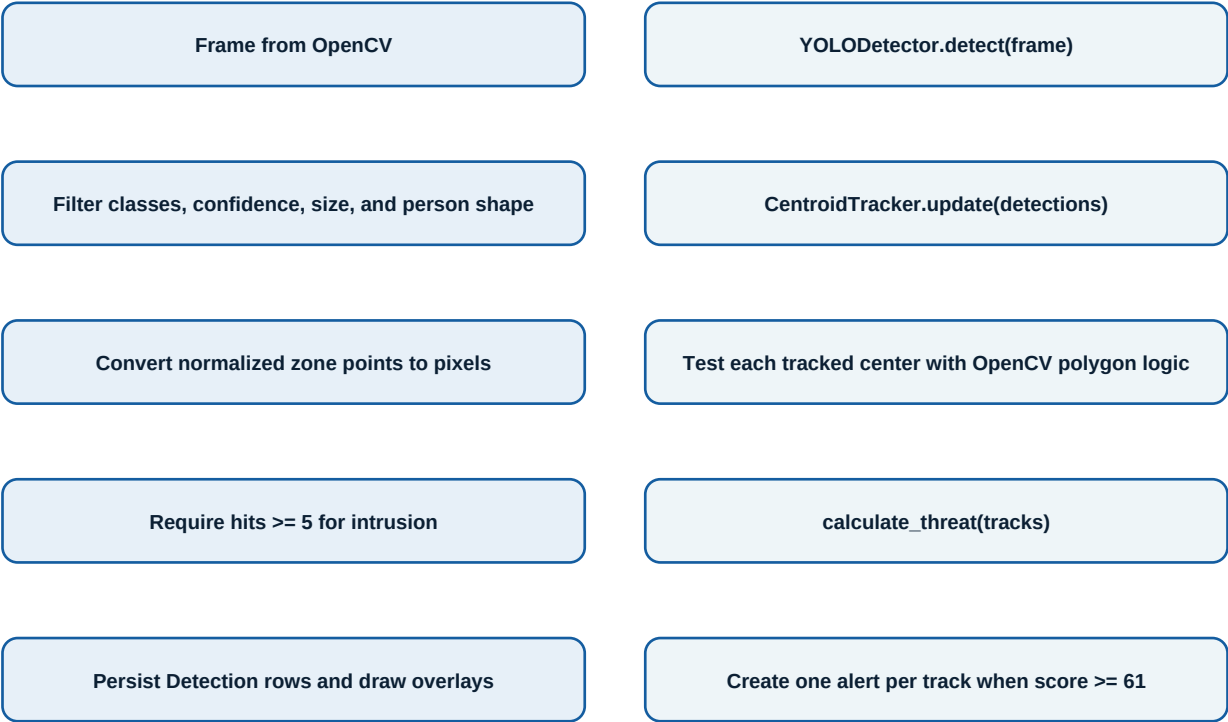
The app creates database tables, adds compatibility columns, resets runtime statuses to offline, creates the evidence directory, includes routers, and mounts evidence as static files. Camera routes are included both with and without `/api`; the active frontend uses the `/api` form.

REST endpoint map

Method and path	Input	Result and frontend use
GET /api/cameras	None	Array of camera records; Dashboard, Cameras, Monitoring, AlertDetailPage.
POST /api/cameras	CameraCreate JSON	Creates a camera; Add Camera form.
POST /api/auth/login	Username and password JSON	Returns an HMAC-signed operator token.
GET /api/auth/me	Bearer token	Validates the current operator session.
POST /api/cameras/upload-video	Multipart file, .mp4	Stores a valid MP4 under data/videos.
DELETE /api/cameras/{id}	Path ID	Deletes the row; Cameras page.
GET /api/alerts	None	Up to 200 newest alerts; Alerts and Dashboard.
GET /api/alerts/{id}	Path ID	One alert; AlertDetailPage.
PATCH /api/alerts/{id}/decision	{decision: confirmed declined}	Stores status and acknowledged_at.
GET /api/events	None	Up to 200 newest events; Events page.
GET /api/detections	None	Up to 500 newest detections; Analytics page.
GET /api/analytics	None	Four aggregate counts; Dashboard and Analytics.
POST /api/streams/start	{camera_id}	Starts a StreamManager worker; Monitoring and supervisor.
POST /api/streams/stop	{camera_id}	Requests worker stop; Monitoring.
GET /api/streams/{id}/status	Path ID	Runtime status and counters; StreamSupervisor.
GET /api/streams/{id}/mjpeg	Bearer header or token query	Multipart JPEG preview; LivePreview and ZoneEditor.
GET/PATCH /api/settings	Settings JSON on PATCH	Reads or updates runtime detection settings.
GET/POST /api/cameras/{id}/zones	Zone JSON on POST	Lists or creates normalized polygon zones.
PUT/DELETE /api/zones/{id}	Zone JSON or path ID	Updates or removes a zone.
PATCH /api/zones/{id}/enabled	{enabled: bool}	Enables or disables a zone.

9. The AI and computer-vision pipeline

The central question is: what happens when one frame reaches the backend? The active StreamManager worker takes a captured frame from its newest-frame queue and calls `DetectionPipeline.process(frame)`.



Data entering and leaving each stage

Stage	Input	Output
Capture reader	VideoCapture source	Raw frame, latest JPEG, one-item queue.
Detector	Raw BGR frame	class, confidence, bbox for supported detections.
Tracker	Current detections and prior boxes	track_id, center, hits, current class and bbox.
Zone logic	Tracked center and pixel polygon	zone_matches and intrusion boolean.
Threat engine	Intruding tracked objects	score, severity, reason.
Storage	Track and alert values plus annotated frame	Detection, Event, Alert rows and evidence JPEG.
Transport	Stored alert or broadcast payload	REST JSON, MJPEG, or WebSocket message.

10. YOLO explained from zero

YOLO means You Only Look Once. It is an object-detection model: it looks at an image and proposes rectangular boxes with a class name and a confidence value. A model's weights are learned numbers stored in a file. In this project the active stream worker passes `ai\_models/yolo/model.pt` to `YOLODetector`. Loading is lazy, so the Ultralytics import and model load occur on the first processing worker that needs them.

What the detector actually accepts

Detector rule	Current value	Meaning
Model file	ai_models/yolo/model.pt	The active stream worker's explicit model path.
Confidence	0.45	YOLO proposals below this confidence are not returned.
Supported classes	person, car, truck, motorcycle, bus, bicycle	Other model classes are discarded.
Minimum box	12 pixels wide and 20 pixels high	Very small proposals are discarded.
Person shape	width / height from 0.15 to 1.25	Unusually shaped person boxes are discarded.

An example detection

```
{
  "class": "person",
  "confidence": 0.87,
  "bbox": [120.0, 80.0, 190.0, 260.0]
}
```

The class says what was detected. Confidence is the model's numeric certainty for this proposal. The bounding box is left, top, right, bottom in pixel coordinates. The detector returns a list of these dictionaries to the tracker. The number is not a legal or security certainty; it is a model score.

The actual detector core

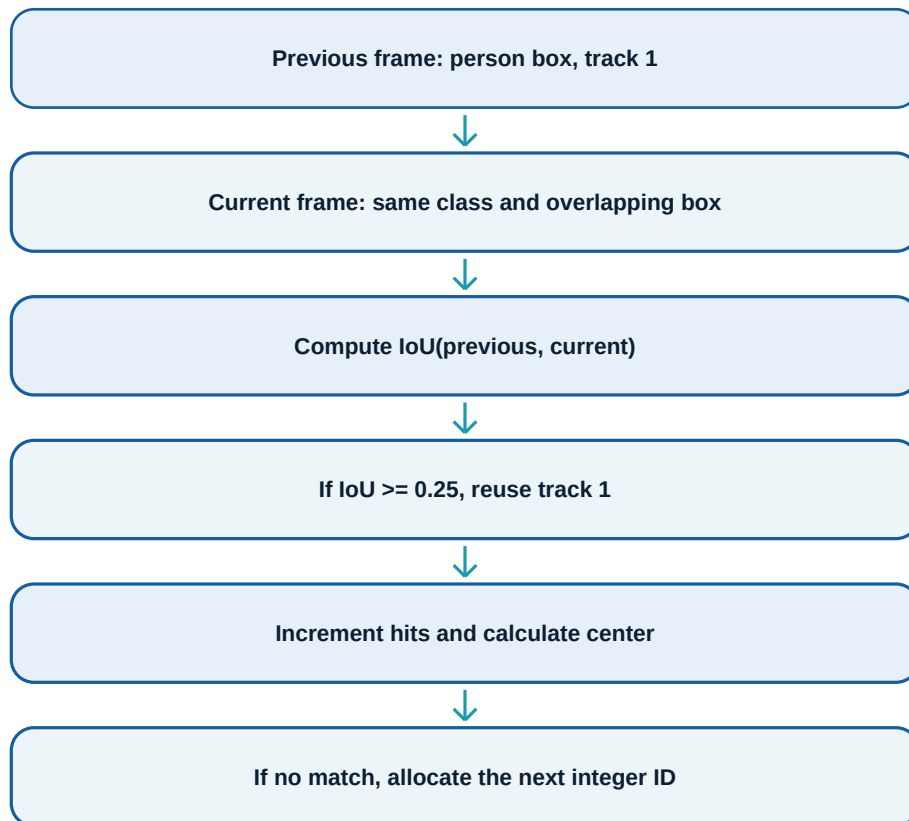
```
self.load()
inference_confidence = min([self.confidence, *self.class_confidences.values()])
results = self.model(frame, conf=inference_confidence, verbose=False)[0]
names = results.names
detections = []
for box in results.bboxes:
    coords = box.xyxy[0].tolist()
    class_id = int(box.cls[0])
    object_class = names[class_id]
    width, height = coords[2] - coords[0], coords[3] - coords[1]
    threshold = self.class_confidences.get(object_class, self.confidence)
    if object_class not in SUPPORTED_CLASSES or float(box.conf[0]) < threshold or width < self.min_width or height < self.min_height:
        continue
    if object_class == "person" and not .15 <= width / max(height, 1) <= 1.25:
        continue
    detections.append({"class": object_class, "confidence": float(box.conf[0]), "bbox": coords})
return detections
```

11. Object tracking and IoU

Detection is frame-by-frame. If a person appears in three frames, detection alone returns three independent boxes. Tracking adds a best-effort identity so the system can say that the same visible object has continued to be present.

Intersection over Union

IoU means Intersection over Union. Imagine two rectangles: the box from the previous frame and the box in the current frame. IoU is the area where they overlap divided by the area covered by either box. A value of 1 means identical boxes; 0 means no overlap.



Exact tracker rules

- `CentroidTracker` uses an IoU threshold of 0.25, despite its historical class name. It matches only detections with the same class.
- Each current detection chooses the unused prior track with the greatest overlap. The best overlap must reach the threshold.
- A new track starts at `next\_id`, which begins at 1. Each matched track gets `hits + 1` and a center equal to the bounding-box midpoint.
- The tracker replaces its track dictionary with the objects seen in the current frame. It does not keep lost tracks for a configurable number of missing frames.

Why hits matter

The pipeline uses hits as persistence. A zone match is not an intrusion until a tracked object has at least five hits. That reduces an immediate one-frame trigger, but it also means a new track or a difficult detection sequence may not reach the threshold.

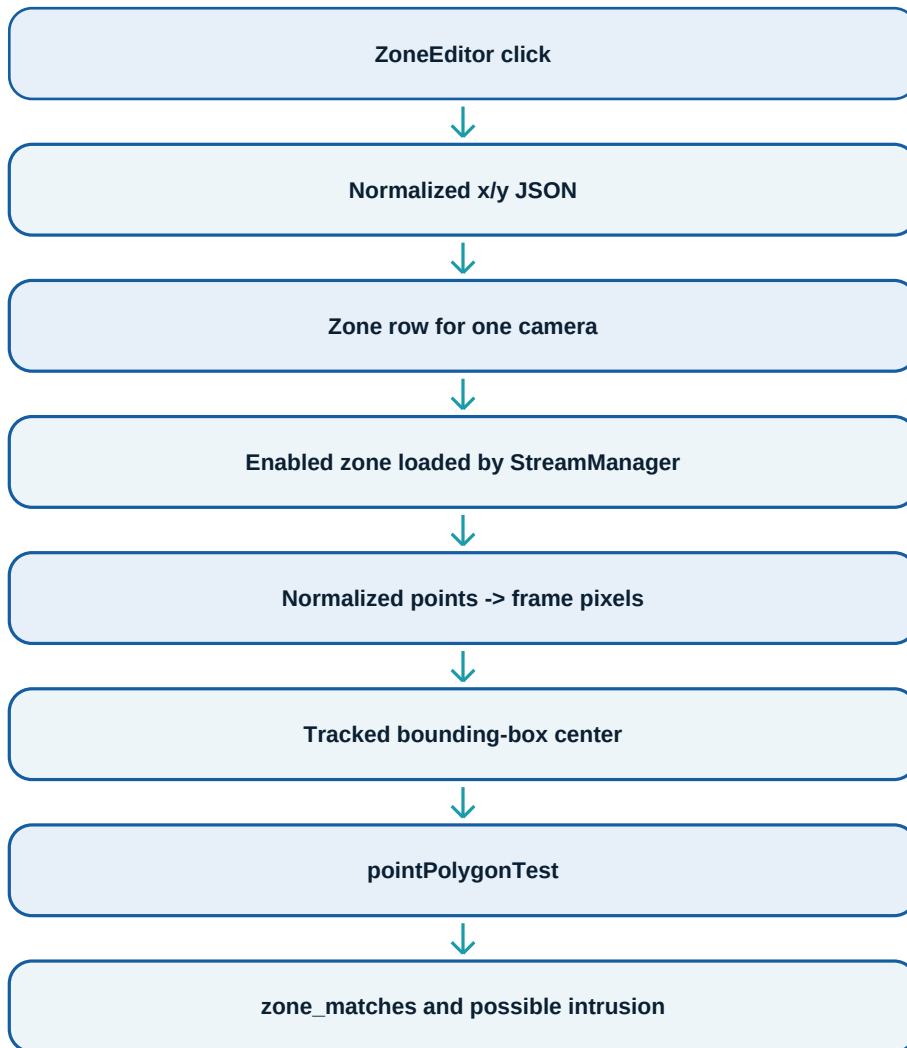
12. Zone detection

A zone is a named polygon attached to one camera. The browser's ZoneEditor records points in normalized coordinates: x and y are each between 0 and 1, so the same shape scales with the actual video dimensions.

```
Normalized browser point: {x: 0.20, y: 0.65}  
Frame size: 1280 x 720  
Pixel point: [round(0.20 * 1280), round(0.65 * 720)] = [256, 468]
```

What the pipeline tests

For every tracked detection, the pipeline takes the box center: $((x1+x2)/2, (y1+y2)/2)$. It converts the camera's enabled zone points to pixel coordinates for the current frame and calls OpenCV `pointPolygonTest`. A point on the boundary counts because the code accepts a result greater than or equal to zero. A polygon needs at least three points; API validation enforces that and also rejects coordinates outside 0 through 1.



Zone types currently accepted

The schemas accept `restricted`, `high_security`, `vehicle_restricted`, and `monitoring`. The current threat pipeline uses a matching enabled zone and its name/type; it does not add a different score for each zone type. The zone type is carried into the stored alert and WebSocket payload.

13. Threat engine

The threat engine is a deterministic rule function, not a second machine-learning model. It receives tracked objects with an `intrusion` flag. If none are intruders, it returns score 0, severity LOW, and the reason `No restricted-zone intrusion`.

Current score table

Rule	Value	When applied
Base person	45	An intruding person.
Base car / motorcycle / bicycle	60 / 60 / 45	The matching intruding class.
Base truck / bus	65 / 65	The matching intruding class.
Persistent object	+20	Any intruder has hits ≥ 5 .
Multiple intruders	+10	More than one object has intrusion=true.
Duration	$+\min(20, \text{int}(\text{duration} * 2))$	The function supports it, but active pipeline calls use the default duration 0.
Cap	100	The final score is never above 100.

Severity thresholds

Score 81 or more is CRITICAL. Score 61 through 80 is HIGH. Score 31 through 60 is MEDIUM. Anything below 31 is LOW. The active StreamManager creates an alert only when a zone intruder exists and the score is at least 61. Therefore a persistent person reaches 65, while a one-hit person stays at 45 and does not create an active-stream alert.



14. Alert system

``create_alert`` is the alert service entry point. It creates an ``Event`` with `event_type`intrusion``, flushes it to obtain its ID, optionally writes the annotated frame as a JPEG, creates an ``Alert``, commits both records, refreshes the alert, and broadcasts a notification payload.

Stored alert information

The Alert row contains the event link, severity, status, message, creation and acknowledgment times, camera ID, track ID, object type, zone, zone type, score, reason, evidence path, and timestamp. New alerts start with status ``active``. The review endpoint accepts exactly ``confirmed`` or ``declined``, updates the status and records ``acknowledged_at``.

Live alert payload

```
db.add(alert)
db.commit()
db.refresh(alert)
camera = db.get(Camera, camera_id)
manager.broadcast_from_sync({
    "type": "alert",
    "data": {
        "id": alert.id, "camera_id": camera_id, "track_id": track_id,
        "object_type": object_type, "zone": zone, "zone_type": zone_type,
        "score": threat["score"], "severity": threat["severity"],
        "reason": threat["reason"], "status": alert.status,
        "alert_name": threat["reason"], "confidence": confidence,
        "camera_name": camera.name if camera else f"Camera {camera_id}",
        "camera_code": camera.camera_code if camera else None,
        "evidence_path": evidence_path,
        "timestamp": now.isoformat(),
    },
})
return alert
```

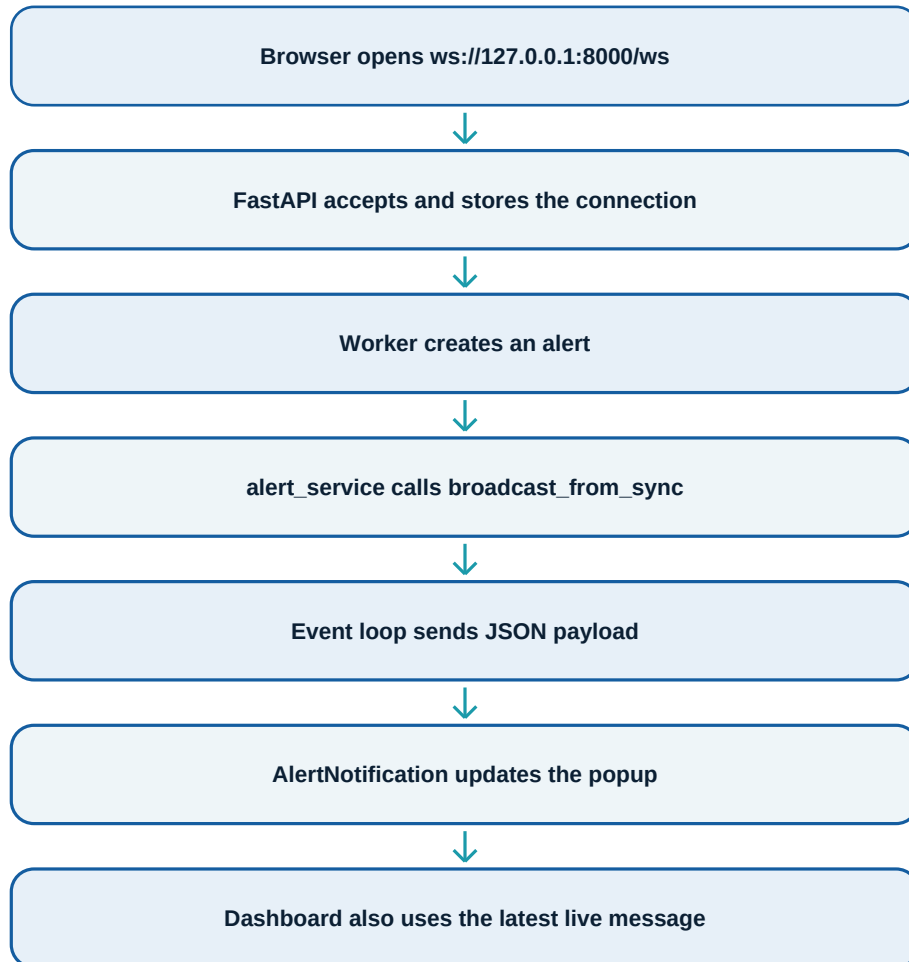
The WebSocket payload adds friendly `camera_name` and `camera_code` values, along with `alert_name`, confidence, `evidence_path`, and an ISO timestamp. REST `AlertResponse` has the stored alert fields but does not include camera name; `AlertDetailPage` therefore fetches cameras separately to resolve the name.

One alert per track

The active stream worker maintains an in-memory ``alerted_tracks`` set. Once a track creates an alert, that track is not alerted again during the current worker lifetime. Restarting a stream creates a fresh set. This is debounce logic for an MVP, not a durable incident deduplication policy.

15. WebSockets

With normal HTTP, the browser asks and the server answers. The request is finished after the response. A WebSocket keeps a connection open so the backend can send a message when an event happens, without waiting for a new browser request.



Actual connection behavior

`frontend/src/hooks/useWebSocket.js` creates a WebSocket, marks it connected on open, parses JSON messages, and reconnects after three seconds when the socket closes. `backend/app/api/websocket.py` accepts connections and waits for client text messages. The server's broadcast path uses the stored event loop to schedule `send\_json` from the processing thread.

Message shape

```

{
  "type": "alert",
  "data": {
    "id": 26, "camera_id": 2, "track_id": 1,
    "object_type": "person", "zone": "Restricted Zone A",
    "score": 65, "severity": "HIGH",
    "reason": "Person entered restricted zone",
    "camera_name": "Real Test Video",
    "evidence_path": "/evidence/alert_...jpg"
  }
}

```

Important

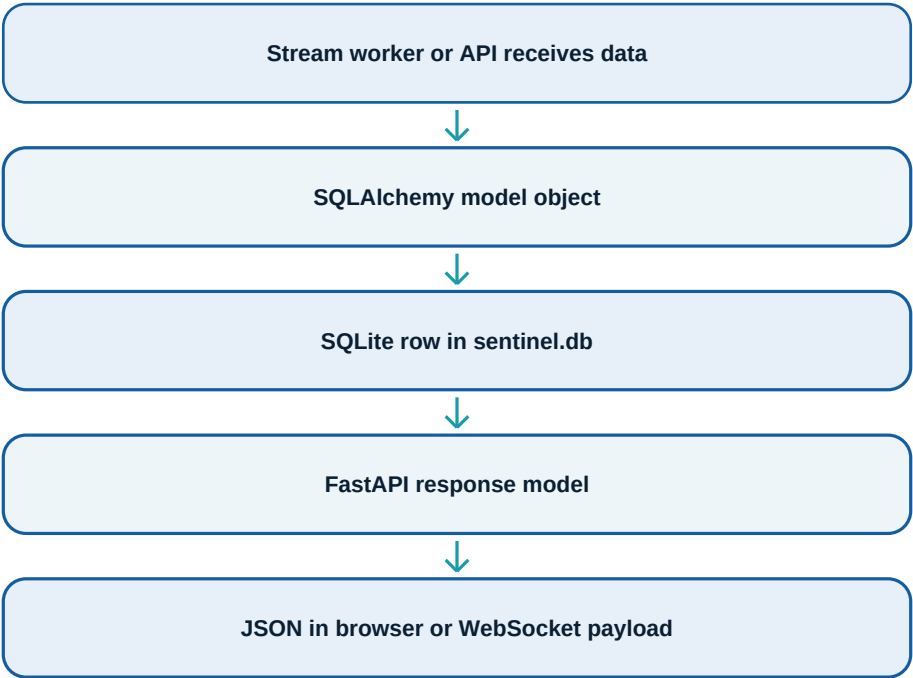
The browser has both a global AlertNotification connection and a Dashboard connection in the active App.jsx. The popup is global; the dashboard's local message is used for its live alert list.

16. Database

SQLite is a relational database stored in a file. A table is like a spreadsheet with a fixed set of columns. A row is one stored record. A primary key uniquely identifies a row. A foreign key points from one table to a related row. SQLAlchemy provides the Python classes and queries used by the backend.

Table	Important columns	Purpose and relationships
cameras	id, camera_code, name, sector, stream_url, location_lat/lng, is_active, source_type, status, created_at	Configured sources. Other tables reference camera_id.
detections	id, camera_id, object_type, confidence, track_id, timestamp	One stored detector/tracker observation.
zones	id, camera_id, name, zone_type, polygon_points, enabled, created_at, updated_at	Camera-specific normalized security polygons.
events	id, camera_id, event_type, severity, description, timestamp	Human-readable incident/event record.
alerts	id, event_id, severity, status, message, acknowledged_at, camera_id, track_id, object_type, zone, score, reason, evidence_path, timestamp	Actionable alert linked to an event and optional evidence.

Data flow



The project-root database path is resolved in ``backend/app/database/database.py``. ``backend/app/main.py`` calls ``Base.metadata.create_all``, adds compatibility columns, and resets runtime camera statuses to offline at startup. The actual database may contain prior demo/test records; this guide documents the schema rather than asserting a particular row count.

17. Video processing

A video file is a sequence of frames recorded at a frame rate, or FPS. `StreamManager.start` resolves relative project paths, checks that a local source exists, stops an existing worker for the same camera, creates shared state, and starts a daemon worker.

Two active threads

Thread	Responsibility
Capture reader	Calls <code>cv2.VideoCapture.read()</code> , loops an MP4 back to frame zero at its source FPS, stores the latest raw JPEG, and puts the newest raw frame into a <code>Queue(maxsize=1)</code> .
Inference worker	Builds zones and <code>DetectionPipeline</code> , consumes queued frames, writes detections, annotates boxes/zones, creates alerts, and stores the latest processed JPEG.

The one-item queue is a deliberate freshness choice. If inference is slower than capture, the reader removes the old queued frame before inserting a newer one. For MP4 sources, reaching the end rewinds rather than ending the worker. For webcam or RTSP sources, failure to return a frame becomes a reader error and the worker goes offline.

Preview lifecycle

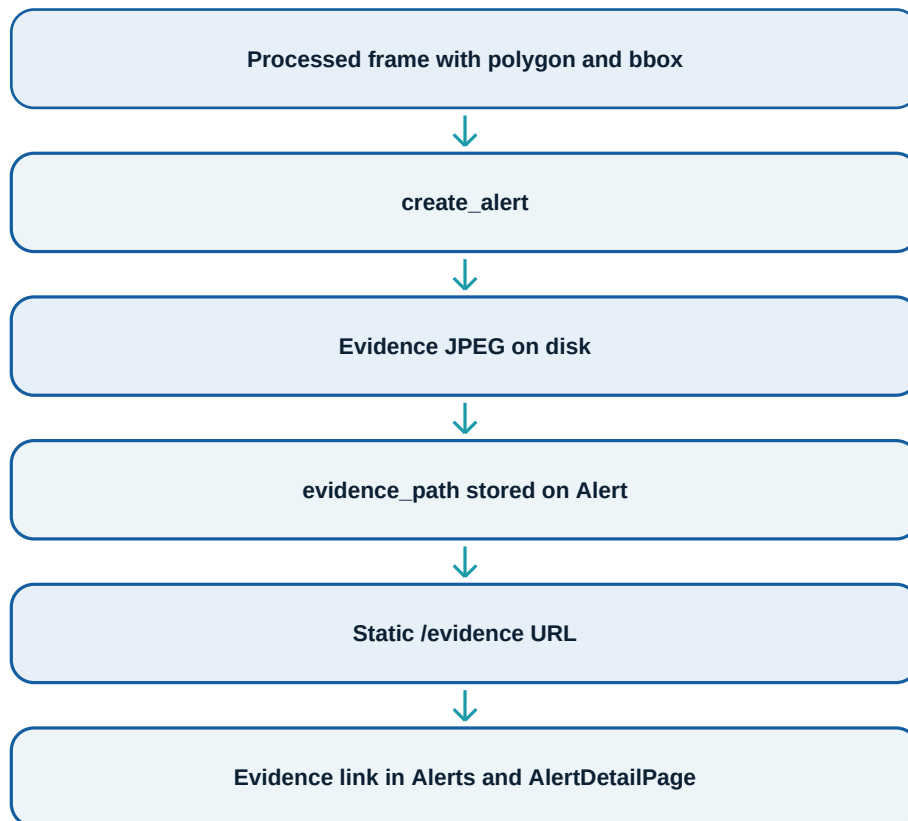
The MJPEG endpoint repeatedly yields the latest in-memory JPEG while status is not offline, error, or stalled. The `status` method reports a stream as stalled if an online stream has not captured a frame for more than 20 seconds. The frontend `LivePreview` appends a `retry` query value after image failures; it does not itself run video processing.

Finite helper

`backend/app/video/frame_processor.py` contains `process_video`, which reads a file to EOF and returns a finite summary. The active HTTP start endpoint uses `StreamManager` instead, so this helper is useful code but not the current browser streaming path.

18. Evidence and event storage

When the active stream worker sees an intruder whose threat score is at least 61, it passes an annotated display frame to `create_alert`. The service creates `data/evidence/` if needed and writes a JPEG named like `alert_YYYYMMDD_HHMMSS_microseconds.jpg`. The Alert row stores a browser path such as `/evidence/alert_....jpg`. FastAPI mounts the directory, so the browser can open `http://127.0.0.1:8000/evidence/`.



What is not stored in the current MVP

- The database does not store every raw video frame.
- The evidence is a JPEG snapshot, not a complete incident video clip.
- Detection rows store class, confidence, track ID, camera ID, and time; they do not store the full bounding box or zone match.
- The frontend Alerts table exposes an evidence link, while the detail page shows the evidence link and review actions.

19. Complete end-to-end example

Scenario: a person enters an enabled polygon for a configured camera. The following is the actual path, not a mock request sequence.

Step	What happens	Relevant implementation
1	A registered video source is started with its camera ID.	POST /api/streams/start -> streams.py -> StreamManager.start.
2	OpenCV reads a frame and the reader puts the newest frame in the queue.	stream_manager.py _capture_reader.
3	The worker passes the frame to YOLO and receives filtered person boxes.	DetectionPipeline -> YOLODetector.detect.
4	The tracker compares the current box with the prior person box.	CentroidTracker.update, IoU >= 0.25.
5	The same track ID gains hits as boxes continue to overlap.	track_id and hits fields.
6	The center of the tracked box falls inside the camera's enabled polygon.	zones_for_frame and point_in_zone.
7	After five hits, intrusion becomes true.	DetectionPipeline.persistence_frames = 5.
8	The person score becomes $45 + 20 = 65$, severity HIGH.	calculate_threat in threat_engine.py.
9	The worker writes a Detection and calls create_alert once for the track.	stream_manager.py and alert_service.py.
10	Event and Alert rows commit; annotated evidence JPEG is written.	sentinel.db and data/evidence/.
11	The alert JSON is broadcast to connected WebSocket clients.	ConnectionManager.broadcast_from_sync.
12	The global popup identifies the alert and links to /alerts/{id}.	AlertNotification in App.jsx.
13	The operator can open the detail page, confirm, and hear local Web Audio.	AlertDetailPage and PATCH decision endpoint.

No fabrication

If YOLO returns no qualifying detections, the tracker and threat path cannot create an intrusion alert. A healthy server or a live JPEG alone does not prove that the model detected a person.

20. Code walkthrough

The most useful code path is the active worker loop. The complete file is larger than a teaching page, so the excerpts below show the logical blocks that connect capture to alert.

A. Start a worker

```
"""Own camera workers and the latest browser-consumable JPEG frame."""

def __init__(self):
    self.streams = {}
    self.lock = Lock()

    @staticmethod
    def resolve_source(source, source_type="video"):
        if source_type == "webcam":
            return int(source) if str(source).isdigit() else source
        if "://" in source:
            return source
        path = Path(source)
        return str(path if path.is_absolute() else PROJECT_ROOT / path)

    def start(self, camera):
        source = camera.stream_url
        if not source:
            raise ValueError("Camera has no video source configured")
```

The source is validated, any prior stream for that camera is asked to stop, a state dictionary is created, and a daemon thread is launched. State holds the latest JPEG, counters, queue, events, and error information shared by the worker and status/preview endpoints.

B. Build the pipeline

```
        with self.lock:
            self.streams[camera.id] = state
            thread.start()

    def stop(self, camera_id):
        with self.lock:
            state = self.streams.get(camera_id)
        if state:
            state["stop"].set()
            if state["status"] == "online":
                state["status"] = "stopping"

    def _worker(self, state, source, source_type):
        camera_id = state["camera_id"]
        db = SessionLocal()
        reader = Thread(target=self._capture_reader, args=(state, source, source_type), daemon=True, name=f"sentinel-reader-{camera_id}")
        reader.start()
        try:
            if not state["reader_ready"].wait(timeout=5):
                state["status"] = "offline"
```

The worker waits for a reader-ready event, loads enabled zones from SQLite, constructs the pipeline with the real model path and confidence 0.45, and marks the camera online. A local set prevents duplicate alerts for one track.

C. Process and persist

```

self._set_camera_status(db, camera_id, "starting")
zones = [{"id": zone.id, "name": zone.name, "zone_type": zone.zone_type, "polygon_points": json.loads(zone.polygon_points)}]
settings = get_runtime_settings()
vehicle_classes = {"car", "truck", "motorcycle", "bus", "bicycle"}
pipeline = DetectionPipeline(
    str(PROJECT_ROOT / "ai_models" / "yolo" / "model.pt"),
    min(settings["detection_confidence"], settings["vehicle_confidence"]),
    zones,
    class_confidences={"person": settings["detection_confidence"], **{item: settings["vehicle_confidence"] for item in vehicle_classes}}
)
state["status"] = "online"
self._set_camera_status(db, camera_id, "online")
alerted_tracks = set()

while not state["stop"].is_set():
    try:
        frame = state["frame_queue"].get(timeout=.5)
    except Empty:
        if state["reader_error"]:
            state["status"] = "offline"
            state["error"] = state["reader_error"]
            break
        continue

    tracks, threat = pipeline.process(frame)
    state["frames_processed"] += 1
    state["detections"] += len(tracks)
    state["tracks"].update(track["track_id"] for track in tracks)
    now = datetime.utcnow()
    for track in tracks:
        db.add(Detection(camera_id=camera_id, track_id=track["track_id"], object_type=track["class"], confidence=track["confidence"], timestamp=now))

```

Each queued frame is processed. Every track is written as a Detection. OpenCV draws the zone, rectangle, class, track ID, and confidence. If an intruder exists, the score passes 61, and its track is new to the alert set, the service is called with the annotated frame. Finally the display is encoded as JPEG for the browser.

D. The pipeline method

```

from .detector import YOLODetector
from .tracker import CentroidTracker
from .zone_detector import zones_for_frame, point_in_zone
from .threat_engine import calculate_threat

class DetectionPipeline:
    def __init__(self, model_path="ai_models/yolo/model.pt", confidence=.45, zones=None, persistence_frames=5, class_confidences=None):
        self.detector = YOLODetector(model_path, confidence, class_confidences=class_confidences)
        self.tracker = CentroidTracker()
        self.zones = zones or []
        self.persistence_frames = persistence_frames

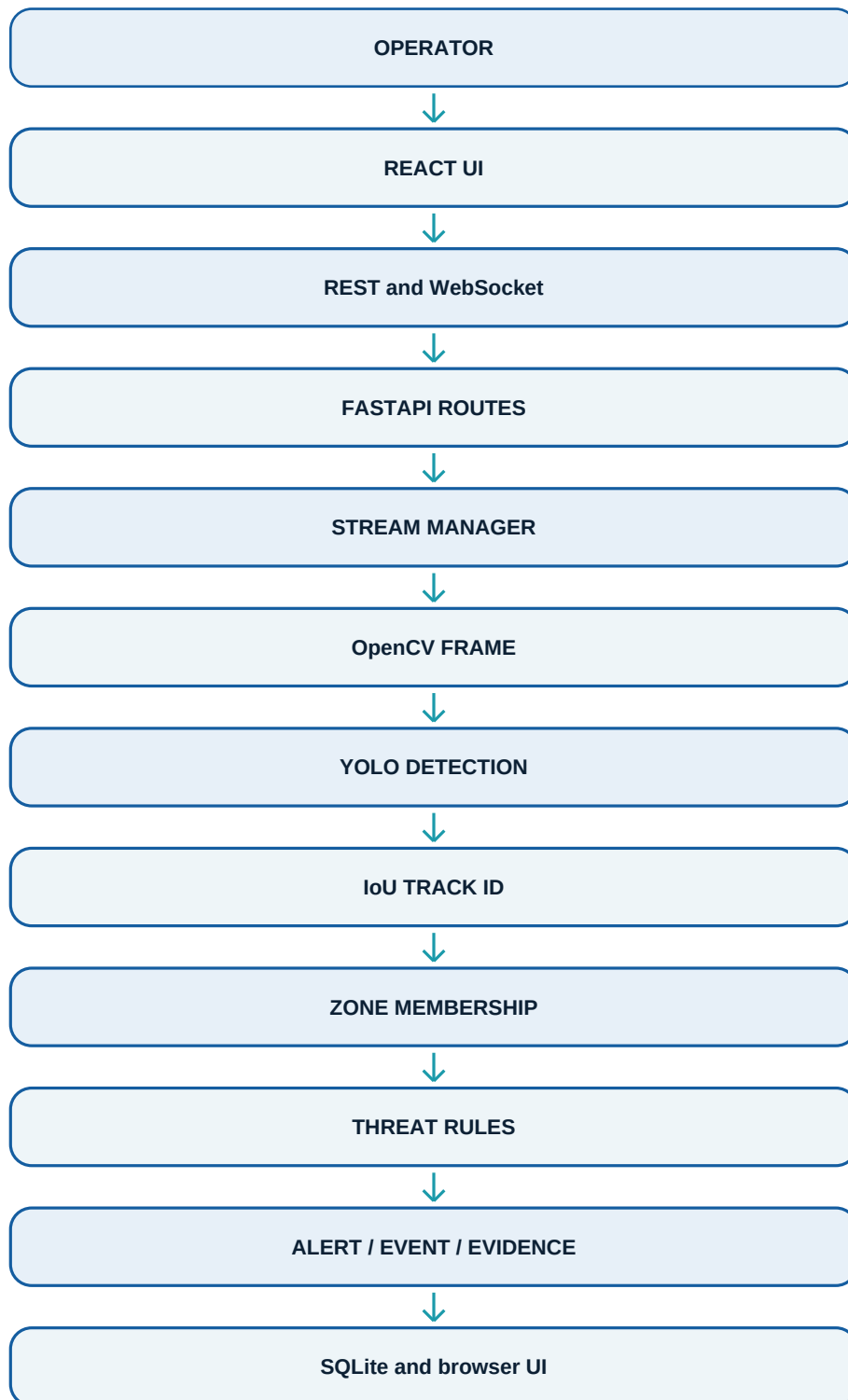
    def process(self, frame):
        tracked = self.tracker.update(self.detector.detect(frame))
        zones = zones_for_frame(self.zones, frame.shape[1], frame.shape[0])
        for item in tracked:
            item["zone_matches"] = [zone for zone in zones if point_in_zone(item["center"], zone)]
            item["intrusion"] = bool(item["zone_matches"] and item["hits"] >= self.persistence_frames)
        return tracked, calculate_threat(tracked)

```

The pipeline composes four small responsibilities: detect, track, convert zones and test centers, then calculate a threat. This separation makes the AI path testable without turning the UI into a second processing engine.

21. How everything connects

Use this mental map when explaining the project or debugging it:



What each arrow means

- Operator -> React: clicks Add Camera, Start Stream, Save Zone, or an alert review action.
- React -> REST: fetch sends JSON or multipart requests to port 8000.
- REST -> backend: FastAPI validates Pydantic input and calls services or the stream manager.

- Stream Manager -> AI: the worker passes raw frames to the pipeline; this does not go through a browser request per frame.
- AI -> storage: detections are inserted continuously; alerts/events/evidence happen only when the threat condition passes.
- Backend -> UI: tables are pulled through REST; new alerts are pushed through WebSocket; frames are streamed as MJPEG.

22. What happens when I start the project?

Backend startup

```
cd C:\...\Sentinel_AI
python -m uvicorn backend.app.main:app --reload
```

The supported root-level command imports `backend.app.main:app`. If the terminal is already inside `backend/`, the `backend/main.py` wrapper supports `python -m uvicorn main:app --reload`. On import, FastAPI is created, CORS is configured for localhost and 127.0.0.1 on port 5173, database tables/compatibility columns are prepared, runtime camera statuses are reset offline, routers are registered, and evidence is mounted.

Frontend startup

```
cd frontend
npm install
npm run dev
```

Vite serves the browser UI, normally at `http://127.0.0.1:5173`. `App.jsx` mounts the router, the global stream supervisor, and the global alert notification. The frontend does not automatically load the YOLO model. The model is lazy-loaded by a processing worker when a stream is started.

Important operational distinction

Creating a camera row does not, by itself, run OpenCV or YOLO. The backend stream API must be called. The active frontend's `StreamSupervisor` also tries to start active sources when they are offline, so the browser session can trigger processing automatically; the backend itself has no database-only auto-start on camera creation.

23. Common terms glossary

Term	Meaning in simple words
API	A contract that lets one program ask another program for work or data.
Backend	The server-side program that owns processing and storage.
Frontend	The browser-side interface used by the operator.
HTTP	The request-and-response protocol used by the REST API.
WebSocket	A connection that stays open for live messages.
JSON	A structured text format for objects, lists, numbers, and strings.
REST	A common style of HTTP API using methods such as GET, POST, PATCH, and DELETE.
CRUD	Create, read, update, delete: the basic database actions.
AI / ML	Software that uses learned patterns or rules to produce an answer.
Computer vision	Extracting information from images or video frames.
YOLO	A real-time object detector that returns boxes, classes, and confidence.
Inference	Running a trained model on a new input frame.
Weights	The learned numeric parameters stored in a model file.
Bounding box	A rectangle around a detected object, represented by four coordinates.
Confidence	The detector's score for how likely its proposed class is correct.
Detection	One object proposal for one frame.
Tracking	Matching detections across frames to maintain an identity.
IoU	Intersection over Union: the overlap ratio between two boxes.
Track ID	An integer label the local tracker assigns to an ongoing object.
FPS	Frames per second, or how quickly a video supplies images.
Frame	One still image from a video sequence.
OpenCV	The Python library used for capture, encoding, drawing, and polygons.
Endpoint / route	A method and URL path that a server exposes.
Request / response	The message sent to a server and the message returned.
Database schema	The tables, columns, and relationships in a database.
SQLite	A relational database stored in a local file.
Primary key	A unique ID for a row.
Foreign key	A column that points to another table's row.
React component	A function that returns part of the browser interface.
React state	A value that can change and cause a component to render again.
Event	A stored occurrence, such as an intrusion, linked to a camera.
Alert	An actionable record created from a high enough threat score.
Evidence	The annotated JPEG snapshot saved for an alert.

24. Why this architecture?

Choice	Practical reason in this MVP
React	The dashboard has many changing values: loading states, alerts, camera selection, and zone points. React state is a simple fit for those changes.
FastAPI	Typed request models, clear route definitions, automatic API docs, and good Python integration make it practical for a local AI service.
OpenCV	It can open local video/camera sources, encode JPEGs, draw overlays, and test polygon membership without another media service.
YOLO	It provides real-time object proposals for common classes and accepts a project model file.
IoU tracking	A small dependency-free matcher is enough for a local demonstration and makes track persistence visible.
WebSocket	An alert can reach the operator immediately instead of waiting for a dashboard refresh.
SQLite	A file database keeps the MVP easy to run locally and easy to inspect.
Mjpeg	A multipart JPEG response is simple for a browser image element and sufficient for a local preview.

Trade-off

These choices optimize for a small local MVP and a clear demo path. They are not automatically the choices for a high-volume, multi-site production deployment.

25. Limitations of the current MVP

These are limitations visible from the implementation, not claims that the system is production-ready.

Area	Current limitation
Detection	Accuracy depends on the trained model, image quality, lighting, camera angle, and the confidence threshold. The code filters to six supported classes and may produce false positives or false negatives.
Tracking	The tracker is a local IoU matcher. It matches same-class boxes and removes tracks not seen in the current frame; it is not a full multi-object tracker with re-identification.
Threat score	Scores are deterministic demo rules. They are not validated security risk assessments, and active stream processing calls duration with its default zero.
Source scale	StreamManager is in-process, with Python threads and in-memory stream state. It is suited to local demonstrations, not an unbounded camera fleet.
Database	SQLite is local and the API lists capped result windows. There is no migration framework or production backup/retention policy in the inspected code.
Authentication	The MVP now has a local HMAC-signed operator token and protects API routes, WebSocket connections, and stream previews. It is not enterprise identity management.
Settings	Settings are runtime values returned by the backend and applied when a new processing worker starts; they are not a durable multi-user configuration store.
Evidence	The system stores an annotated JPEG, not a full clip or tamper-proof evidence package.
Streaming	The preview is a latest-frame MJPEG path. There is no adaptive bitrate, stream recording, or production media gateway.
Frontend wiring	App.jsx is the active UI, while older page and service files coexist and can confuse maintenance if they are treated as active.

26. Future improvements

The following are reasonable next steps, but they are not part of the current MVP unless stated above.

Future / not implemented	Why it would help
Enterprise authentication and roles	Replace the local operator token with an identity provider, durable users, permissions, and audit trails.
Persistent configuration for thresholds	Let operators change confidence and threat policy through a validated backend API.
Stronger tracking	Use a mature tracker with missed-frame tolerance and re-identification for crowded scenes.
Production media service	Move capture and delivery to a managed streaming architecture with health, buffering, and recording policies.
Queue or worker service	Isolate many camera pipelines and recover workers without tying all work to one Python process.
PostgreSQL and migrations	Support concurrent production usage, schema history, and safer backups.
Evidence retention and audit	Keep incident clips, operator decisions, audit logs, and integrity metadata.
Model evaluation	Measure precision, recall, latency, and per-camera performance on labeled border scenes.
Alert investigation workflow	Add operator notes, assignment, status history, and direct navigation from alert tables.
Observability	Add structured logs, metrics, worker heartbeats, and clear model/source health dashboards.

Status boundary

A future improvement belongs in a proposal or backlog, not in a claim about the current system. During an SIH demonstration, describe it as a next step.

27. SIH demo explanation

A reliable demo follows the path the repository actually supports.

Demo step	What to show
1	Start the backend from the repository root and open the frontend.
2	Use the existing configured camera or add a real MP4 through Add Camera; do not describe a mock camera as a detection.
3	Open Monitoring and select the source.
4	Start the stream. The frontend may also ask the StreamSupervisor to keep active sources running.
5	Show the processed MJPEG frame with model boxes, track ID labels, and zone outline when the source/model produce them.
6	When a qualifying object remains in an enabled zone for five hits and the score reaches 61, show the generated alert.
7	Open the alert notification or /alerts, then open the evidence JPEG.
8	Open the alert detail page and explain confirm versus decline. Confirm uses a laptop speaker Web Audio alarm; decline does not.
9	Show Events and Analytics to connect the alert to stored event/detection totals.
10	If asked about missing alerts, check model path, source status, zone shape, confidence, track persistence, and logs rather than claiming a detection.

Demo truth

A running preview proves capture and delivery. It does not prove an object was detected. Use the backend counters, detection rows, alert row, evidence file, and WebSocket message as separate evidence.

28. Explain it in 60 seconds

Sentinel AI is a local border-surveillance MVP. An operator signs in to the protected React dashboard, which communicates with a FastAPI backend. The backend receives video through OpenCV and processes frames. A YOLO model looks for supported objects such as people and vehicles. A small IoU tracker keeps a track ID when boxes overlap across frames. For each camera, the operator can draw a normalized polygon zone. When an object stays in that zone for five tracked hits, deterministic threat rules calculate a score. A score of 61 or more creates an intrusion event and alert, saves an annotated evidence JPEG, stores the data in SQLite, and broadcasts a JSON message over WebSocket. The dashboard shows the live processed preview, alert notification, evidence, events, analytics, review controls, and runtime settings.



29. If a judge asks how it works

Question	Code-accurate answer
How does detection work?	The stream worker calls YOLO through YOLODetector, filters confidence, supported classes, minimum dimensions, and person aspect ratio, then returns boxes and confidence.
Why YOLO?	It provides a practical real-time object-detection interface and can load the project's model.pt. The current guide does not claim a benchmark that is not stored in the repo.
How does tracking work?	CentroidTracker actually uses same-class bounding-box IoU. If best IoU is at least 0.25, it reuses the track ID and increments hits; otherwise it creates the next ID.
How do you identify a threat?	The track center must match an enabled camera polygon for at least five hits, then the deterministic threat score must be at least 61 for the active worker to create an alert.
How does the restricted zone work?	The UI stores x/y points normalized from 0 to 1. The backend scales them to the current frame and tests the tracked bounding-box center with OpenCV pointPolygonTest.
How are alerts generated?	StreamManager calls create_alert once per track after the threshold. The service creates an Event and Alert, writes an evidence JPEG, commits, then broadcasts.
How does frontend communication work?	fetch calls REST endpoints at http://127.0.0.1:8000. useWebSocket listens at ws://127.0.0.1:8000/ws for alert messages. MJPEG supplies preview frames.
Why FastAPI?	It gives the Python project typed request schemas, route organization, automatic API documentation, and WebSocket support.
Where is data stored?	The project-root sentinel.db stores records. Evidence snapshots are under data/evidence and are served at /evidence.
How are multiple detections handled?	YOLO returns a list; the tracker updates each list item; each track gets a Detection row. The threat engine can add a multiple-intruder score bonus.
What happens with low confidence?	YOLO proposals below 0.45 are not returned by the active detector, so no track or alert can be created from those proposals.
What are the limitations?	The small IoU tracker has no re-identification, scores are deterministic demo rules, SQLite and in-process threads limit scale, local authentication has no roles, and runtime settings reset with the process.
How could it scale?	Separate camera workers into a managed queue/service, use stronger tracking, production media delivery, PostgreSQL plus migrations, authentication, observability, and evaluated model deployment.
What if a camera disconnects?	The reader records an error when it cannot open or returns no frames. The worker goes offline; status can become stalled after 20 seconds without a frame. The frontend supervisor can retry active sources.

Final source map

For implementation questions, start with ``backend/app/main.py``, ``backend/app/api/``, ``backend/app/video/stream_manager.py``, ``backend/app/ai/``, ``backend/app/services/alert_service.py``, ``backend/app/database/models.py``, and ``frontend/src/App.jsx``. Those files define the active path documented here.

End of guide

The PDF was generated from the current repository source. It intentionally records inactive files and unimplemented features so a reader can distinguish the real MVP from future architecture.